Generic vs. non-generic Poseidon2 and Fiat–Shamir

Benchmark report, `prover-ray/crypto/genericbench`

28 October 2026

1 Summary

- The fully generic Poseidon2 code is about **7–10× slower** than the non-generic baseline on KoalaBear and about **3× slower** on MamaBear. KoalaBear's gap is larger because its baseline uses gnark-crypto's AVX-512 assembly.
- The code the prover runs today (`current`: the generic hasher and transcript over gnark-crypto's compression) is **on par with the baseline** (within 2%). Production is not a□ected.
- The root cause of the generic overhead is Go's implementation of generics: field operations are indirect calls that cannot be inlined, and they force local variables onto the heap. The generic permutation made 222–340 heap allocations per call.
- Branching to concrete KoalaBear code inside the permutation's hot loops removes all allocations from the permutation and makes it about **2.5× faster**, but it stays about **3.5× slower** than baseline.
- Delegating the whole KoalaBear permutation to gnark-crypto's assembly (`generic-asm`) **matches the baseline** for the permutation itself. Batched `koalabear.Vector` operations inside the hot loops (`generic-vec`) are **about 20× slower** and are a dead end.
- The remaining gap, which is large under parallel load (`CompressParallel`), comes from **2 heap allocations per compression** in the generic `Compress` function. It is the next thing to fix.

2 Setup

Machine. Intel Xeon Platinum 8488C, 192 cores, AVX-512 available (so the KoalaBear baseline runs gnark-crypto's assembly path), Linux amd64, Go 1.27.1, gnark-crypto v0.22.0. Nothing else ran on the machine during the benchmarks.

Implementations compared. Each benchmark name ends in `impl=...`:

- `baseline`: non-generic code. For KoalaBear, gnark-crypto's permutation and compression, and the pre-generic MD hasher and transcript (frozen in `legacy_test.go`). For MamaBear, which never had a non-generic hasher, gnark-crypto's permutation under the generic hasher and transcript.
- `current` (KoalaBear only): what the prover runs today.
- `generic`: everything generic, permutation included. In Tables 3 and 4 it includes the concrete KoalaBear branches (Section 5).
- `generic-vec`, `generic-asm` (KoalaBear only): the two alternative branches of Section 6.

Runs. Three runs, each preceded by `TestAgreement`, which checks that all implementations produce bit-identical outputs. It passed every time.

Run	Code	Sizes	Samples
1 (Tables 1, 2)	`3ce6cc6e5`, fully generic	`full`, minus FS rounds=4096/update=16384	6×500 ms
2 (Table 3)	scalar KoalaBear branches	same as run 1, KoalaBear only	6×500 ms
3 (Table 4)	`6b3d84ad3`, all three paths	`medium`, KoalaBear only	6×200 ms

The single skipped FiatShamir case took about 40 s per operation in the generic code and dominated the runtime. Tables report the median of 6 samples. Unless noted otherwise, every di□erence against the baseline is statistically significant according to `benchstat` (p = 0.002). The exceptions are: `current` on MDHasher $n = 2^{20}$ in run 1 (p = 0.065), `current` on FiatShamir rounds=16/update=16384 in run 2 (p = 0.093), and `current` on FiatShamir rounds=16/update=8 in run 3 (p = 0.39). All three are di□erences of under 2% that `benchstat` could not distinguish from noise.

3 Background: what is benchmarked

All the benchmarks are built on the **Poseidon2 permutation**, the core of the Poseidon2 hash. It takes a fixed-size state of 16 or 24 field elements and scrambles it into another state of the same size. It is reversible, hence "permutation". It runs 27 rounds (6 full, 21 partial), each made of three steps:

1. **Add round keys**: add fixed constants derived from a seed.
2. **S-box**: cube each element, $x \mapsto x^3$. This is the nonlinear step that gives the hash its security. Full rounds cube every element; partial rounds cube only the first.
3. **Linear mixing**: multiply the state by a fixed matrix, so every element a□ects every other. Full rounds use the "external" matrix built from 4×4 blocks (M4); partial rounds use the cheaper "internal" matrix $\mathbf{1}\mathbf{1}^T + D$.

The other benchmark families are built on the permutation:

- **Compress**: two 8-element blocks form a width-16 state, which is permuted; the second block is then added back (feed-forward). **CompressParallel** runs the same with one goroutine per core (192).
- **MDHasher**: hashes n elements, 8 at a time, with one compression per chunk.
- **FiatShamir**: a protocol-shaped transcript. Each round absorbs upd elements, samples a digest, samples query indices and derives a named challenge.

The permutation dominates every benchmark, so its speed carries through to all of them.

4 Results 1: fully generic code

Table 1: KoalaBear, fully generic code (before any branching). Ratios are time relative to `baseline`.

Benchmark	baseline	`generic`	ratio	`current`	ratio	allocs `baseline`	allocs `generic`	allocs `current`
Permutation (width=16)	570 ns	5.18 µs	9.09×	–	–	0	222	–
Permutation (width=24)	706 ns	7.43 µs	10.5×	–	–	0	298	–
Compress	585 ns	5.22 µs	8.91×	–	–	0	224	–
CompressParallel	4.31 ns	967 ns	224×	–	–	0	224	–
MDHasher (n=8)	733 ns	4.96 µs	6.77×	627 ns	0.86×	5	225	1
MDHasher (n=64)	4.99 µs	39.3 µs	7.87×	4.88 µs	0.98×	5	1 793	1
MDHasher (n=4096)	308 µs	2.52 ms	8.20×	311 µs	1.01×	5	114 689	1
MDHasher (n=65536)	4.91 ms	40.1 ms	8.16×	4.96 ms	1.01×	5	1 835 009	1
MDHasher (n=1048576)	78.2 ms	626 ms	8.01×	78.8 ms	1.01×	5	29 360 129	1
FiatShamir (rounds=16, upd=8)	388 µs	2.79 ms	7.19×	394 µs	1.02×	254	126 171	731
FiatShamir (rounds=16, upd=1024)	1.61 ms	12.8 ms	7.93×	1.63 ms	1.01×	254	581 342	731
FiatShamir (rounds=16, upd=16384)	20 ms	164 ms	8.17×	20.2 ms	1.01×	255	7 462 669	732
FiatShamir (rounds=256, upd=8)	6.18 ms	44.6 ms	7.22×	6.31 ms	1.02×	3 849	2 018 586	11 526
FiatShamir (rounds=256, upd=1024)	25.8 ms	204 ms	7.92×	26 ms	1.01×	3 850	9 301 340	11 528
FiatShamir (rounds=256, upd=16384)	320 ms	2.61 s	8.14×	323 ms	1.01×	3 874	119 402 596	11 550
FiatShamir (rounds=4096, upd=8)	99.5 ms	708 ms	7.12×	101 ms	1.02×	65 518	32 301 300	188 400
FiatShamir (rounds=4096, upd=1024)	412 ms	3.2 s	7.77×	417 ms	1.01×	65 536	148 825 042	188 420

Table 2: MamaBear, fully generic code. The MamaBear baseline is gnark-crypto's (non-assembly) permutation under the generic hasher.

Benchmark	baseline	`generic`	ratio	allocs `baseline`	allocs `generic`
Permutation (width=16)	2.09 µs	7.27 µs	3.48×	0	250
Permutation (width=24)	3.39 µs	10.1 µs	2.98×	0	340
Compress	2.2 µs	7.26 µs	3.30×	2	252
CompressParallel	211 ns	1.42 µs	6.72×	2	252
MDHasher (n=8)	2.12 µs	6.71 µs	3.16×	3	253
MDHasher (n=64)	17.2 µs	53.7 µs	3.13×	17	2 017
MDHasher (n=4096)	1.13 ms	3.45 ms	3.06×	1 025	129 025
MDHasher (n=65536)	18 ms	54.3 ms	3.01×	16 385	2 064 385
MDHasher (n=1048576)	290 ms	820 ms	2.83×	262 145	33 030 145
FiatShamir (rounds=16, upd=8)	1.29 ms	3.82 ms	2.95×	1 788	141 789
FiatShamir (rounds=16, upd=1024)	5.78 ms	17.5 ms	3.03×	5 852	653 858
FiatShamir (rounds=16, upd=16384)	73.6 ms	223 ms	3.04×	67 300	8 395 370
FiatShamir (rounds=256, upd=8)	20.6 ms	61.2 ms	2.97×	28 439	2 268 465
FiatShamir (rounds=256, upd=1024)	92.5 ms	279 ms	3.02×	93 472	10 461 576
FiatShamir (rounds=256, upd=16384)	1.17 s	3.56 s	3.03×	1 076 630	134 325 810
FiatShamir (rounds=4096, upd=8)	330 ms	945 ms	2.86×	458 573	36 298 855
FiatShamir (rounds=4096, upd=1024)	1.47 s	4.31 s	2.93×	1 499 070	167 388 320

How to read the tables. Each ratio is time relative to `baseline`: 1× is the same speed, 9× means nine times as long, and below 1× is faster. The allocs columns give heap allocations per operation.

Findings.

- KoalaBear `generic` is 6.8–10.5× slower than baseline, and MamaBear `generic` 2.8–3.5×. Without assembly on the MamaBear side, the MamaBear figure is the better measure of the cost of genericity itself.
- `current` is on par with baseline: from 14% faster (MDHasher, n = 8) to about 2% slower (FiatShamir). It also makes 80% fewer allocations in MDHasher, but about 2.9× more in FiatShamir. The absolute numbers are small (731 vs. 254 at 16 rounds), but this may deserve a look.
- The generic code allocates heavily: 222–340 allocations per permutation, about 28 per absorbed element in the KoalaBear MDHasher, and up to 167 million per FiatShamir operation.

Why the generic code is slow. Go does not compile a generic function separately for every type. It compiles it once per "GC shape" (here, every `[1]uint32`-shaped field shares the code) and passes a dictionary of the type's methods. So inside `sBox`, `matMulM4` and the other helpers, each `PT(x).Mul`, `PT(x).Add`, ... is an *indirect call* through that dictionary. The compiler cannot inline it, and it cannot see what the callee does with the pointers it receives, so it moves the operands to the heap. That causes both the per-operation call overhead and the hundreds of allocations per permutation.

5 Results 2: branching to concrete KoalaBear code in the hot loops

Each hot helper of the permutation (`sBox`, `addRoundKey`, `matMulExternal` with M4, `matMulInternal`) now starts with `if h.kb { ...; return }`. The flag `kb` is computed once in `NewPermutation` by a type check. The KoalaBear branch reinterprets its operands as `[]koalabear.Element` (via `unsafe`, without copying) and runs concrete loops that the compiler can inline and keep on the stack. The check runs once per helper call; putting it inside the per-element loops would only add work. Other fields keep the generic code.

Table 3: KoalaBear with concrete `koalabear.Element` branches in the permutation's hot loops (`KoalaBearScalar`).

Benchmark	baseline	`generic`	ratio	`current`	ratio	allocs `baseline`	allocs `generic`
Permutation (width=16)	570 ns	2.03 µs	3.56×	–	–	0	0
Permutation (width=24)	705 ns	2.85 µs	4.05×	–	–	0	0
Compress	585 ns	2.06 µs	3.52×	–	–	0	2
CompressParallel	4.28 ns	156 ns	36.5×	–	–	0	2
MDHasher (n=8)	728 ns	2.09 µs	2.87×	626 ns	0.86×	5	3
MDHasher (n=64)	4.95 µs	16.6 µs	3.35×	4.88 µs	0.98×	5	17
MDHasher (n=4096)	309 µs	1.06 ms	3.43×	312 µs	1.01×	5	1 025
MDHasher (n=65536)	4.88 ms	16.9 ms	3.46×	4.94 ms	1.01×	5	16 385
MDHasher (n=1048576)	78.7 ms	270 ms	3.43×	79.6 ms	1.01×	5	262 145
FiatShamir (rounds=16, upd=8)	388 µs	1.22 ms	3.15×	396 µs	1.02×	254	1 850
FiatShamir (rounds=16, upd=1024)	1.61 ms	5.4 ms	3.35×	1.63 ms	1.01×	254	5 914
FiatShamir (rounds=16, upd=16384)	20.1 ms	68.9 ms	3.43×	20.3 ms	1.01×	256	67 360
FiatShamir (rounds=256, upd=8)	6.2 ms	19.5 ms	3.15×	6.31 ms	1.02×	3 849	29 447
FiatShamir (rounds=256, upd=1024)	25.7 ms	86.9 ms	3.38×	26.1 ms	1.01×	3 850	94 478
FiatShamir (rounds=256, upd=16384)	321 ms	1.1 s	3.42×	324 ms	1.01×	3 880	1 077 606
FiatShamir (rounds=4096, upd=8)	98.6 ms	311 ms	3.16×	101 ms	1.02×	65 518	475 147
FiatShamir (rounds=4096, upd=1024)	413 ms	1.38 s	3.34×	417 ms	1.01×	65 542	1 515 612

Findings. The permutation is now allocation-free and about 2.5× faster than fully generic, but still 3.6–4.0× slower than baseline. The remaining gap is the AVX-512 assembly: one AVX-512 instruction processes 16 elements at once, which is exactly one width-16 state, and gnark-crypto's assembly keeps the whole state in registers for all 27 rounds. Scalar Go code cannot do that. The remaining allocations (2 per compression) are analysed in Section 7.

6 Results 3: vector operations vs. assembly

Two ways to close the assembly gap were implemented side by side. The choice is made with `SetKoalaBearPath`:

- `KoalaBearScalar` (default, column `generic`): the concrete loops of Section 5.
- `KoalaBearVector` (`generic-vec`): the same branches, but the round keys, the full S-box layer and the internal matrix use gnark-crypto's batched `koalabear.Vector` operations (`Add`, `Mul`, `Sum`), which run AVX-512 loops over

slices. M4 and the external matrix stay scalar because they mix neighbouring elements. For width 24, the last 8 elements fall back to scalar code because the vector operations work in blocks of 16.

- `KoalaBearAsm` (`generic-asm`): the branch sits at the top of `Permute` and hands the whole permutation to gnark-crypto's AVX-512 function.

Table 4: KoalaBear, the three code paths side by side (`medium` scale). `generic` is the scalar branch.

Benchmark	baseline	`generic`	ratio	`generic-vec`	ratio	`generic-asm`	ratio	`current`	ratio	allocs `generic-asm`
Permutation (width=16)	570 ns	1.92 µs	3.37×	11.6 µs	20.4×	573 ns	1.01×	–	–	0
Permutation (width=24)	706 ns	2.63 µs	3.72×	12.3 µs	17.4×	706 ns	1.00×	–	–	0
Compress	586 ns	2.03 µs	3.46×	11.7 µs	19.9×	656 ns	1.12×	–	–	2
CompressParallel	4.4 ns	170 ns	38.7×	471 ns	107×	73.2 ns	16.6×	–	–	2
MDHasher (n=64)	4.97 µs	16.2 µs	3.26×	93.4 µs	18.8×	5.3 µs	1.07×	4.88 µs	0.98×	17
MDHasher (n=4096)	308 µs	1.03 ms	3.35×	5.98 ms	19.4×	336 µs	1.09×	312 µs	1.01×	1 025
MDHasher (n=65536)	4.92 ms	16.6 ms	3.38×	95.5 ms	19.4×	5.34 ms	1.09×	4.96 ms	1.01×	16 385
FiatShamir (rounds=16, upd=8)	133 µs	381 µs	2.86×	2.08 ms	15.6×	141 µs	1.06×	133 µs	1.00×	698
FiatShamir (rounds=16, upd=1024)	1.35 ms	4.5 ms	3.33×	25.8 ms	19.1×	1.5 ms	1.10×	1.37 ms	1.01×	4 762
FiatShamir (rounds=256, upd=8)	2.11 ms	6.1 ms	2.89×	33.2 ms	15.7×	2.26 ms	1.07×	2.13 ms	1.01×	11 013
FiatShamir (rounds=256, upd=1024)	21.7 ms	71.9 ms	3.31×	413 ms	19.0×	23.7 ms	1.09×	21.9 ms	1.01×	76 043

Findings.

- `generic-asm` matches the baseline permutation (1.00–1.01×). The Compress, MDHasher and FiatShamir benchmarks remain 6–12% slower than baseline. That overhead comes from the generic code around the permutation.
- `generic-vec` is about 20× slower than baseline, and about 6× slower than the scalar branch it was meant to improve. A CPU profile shows almost all the time inside the vector routines (75% in the 21 partial rounds), at about 150 ns per call on 16 elements. The vector routines are designed for long arrays; on a 16-element state the cost of each call outweighs the work. The likely cause is the penalty the CPU pays when switching between AVX-512 and ordinary instructions, but this has not been confirmed.

Why the two cannot be combined in one permutation. Both paths live in the same codebase and are compiled into the same binary; nothing conflicts. A given permutation can use only one of them, though: the assembly function performs all 27 rounds in a single call, so once `Permute` hands it the state, none of the per-step helpers run and there is nowhere for a vector branch to apply. gnark-crypto does not expose the individual rounds of its assembly. This is the underlying trade-o□: branching as low as possible and matching the baseline's speed pull in opposite directions, because the fast code exists only as one whole-permutation block.

7 Why CompressParallel is so slow

Even with the assembly permutation, `CompressParallel` is 16.6× slower than baseline (73 ns vs. 4.4 ns per compression across 192 cores). A memory profile attributes all the allocations to two lines of the generic `Compress` (`crypto/poseidon2/hasher.go`):

```
var x [CompressionWidth]T // line 26: the 16-element working state
res := block // line 32: the 8-element result
```

These are fixed-size local arrays and should stay on the stack, as they do in gnark-crypto's almost identical `CompressPoseidon2`. They escape for the reason given in Section 4: `x` is passed to `perm.Permute` and `res` to `PT(&res[i]).Add`, both indirect calls the compiler cannot see through, so it assumes the pointers may be kept and moves both arrays to the heap.

- **Single-threaded**, two small allocations per compression cost little: 656 ns vs. 586 ns.
- **On 192 cores**, the baseline allocates nothing and scales perfectly. The generic version makes about 2 allocations every few nanoseconds across all cores. The allocator, the garbage collector and memory tra□c between cores become the bottleneck. The same two allocations account for the 16 385 allocations of MDHasher at n = 65 536.

8 Recommendations

1. **Make `KoalaBearAsm` the default** when the CPU supports AVX-512 and the parameters are 6 full and 21 partial rounds; otherwise fall back to the scalar branch.
2. **Remove the two allocations in `Compress`** by applying the same KoalaBear branch there: reinterpret `x` and `res` as KoalaBear arrays and do the feed-forward with direct calls. A per-permutation scratch bu□er would not work, as it would break concurrent use.
3. **Drop `KoalaBearVector`**, or keep it only as a recorded negative result.

4. **For MamaBear** (about 3×, with no assembly available), the overhead is purely the generic code. Closing it means removing the per-operation allocations, for example with concrete branches like KoalaBear's.
5. Look into the higher allocation count of `current` in FiatShamir (about 2.9× baseline).

A Reproducing

From `prover-ray/`, with `benchstat` installed (`go install golang.org/x/perf/cmd/benchstat@latest`):

```
# all fields, medium scale (a few minutes)
scripts/bench-generic-poseidon.sh medium <outdir>

# KoalaBear only, as in run 3 (about 80 s)
go test ./crypto/genericbench/ -run TestAgreement
GENERIC_BENCH_SCALE=medium go test ./crypto/genericbench/ -run '^$' \
  -bench '/field=koalabear/' -benchmem -count 6 -benchtime 200ms > raw.txt
benchstat -col /impl raw.txt
```